МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ
Кафедра интеллектуальных информационных технологий

Отчёт по лабораторной работе №4

Специальность АС-67/24

Выполнил:
Тюрина Т. А.
Студент 3 курса
Группы АС-67
Проверил:
Крощенко А. А.
доц. ИИТ,

Брест 2026

Цель работы: построить, обучить и оценить многослойный перцептрон (MLP) для решения задачи классификации.

Ход работы

Вариант 9

Общий план для всех вариантов:

1. **Импорт библиотек и подготовка данных**

- импортируйте `torch`, `torch.nn`, `torch.optim`, а также `sklearn` для загрузки данных и их предобработки;
- загрузите датасет, выполните **стандартизацию** (`StandardScaler`) и **кодирование** признаков;
- разделите данные на обучающую и тестовую выборки;
- **преобразуйте данные (признаки и метки) в тензоры PyTorch: `torch.tensor(X_train, dtype=torch.float32)`.**

Код:

```
# Импорт библиотек для работы с данными и построения нейронной сети

import numpy as np
import pandas as pd

import torch
import torch.nn as nn
import torch.optim as optim

from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score


# Загрузка датасета Glass Identification из CSV-файла

data = pd.read_csv('/content/glass.csv')

X = data.drop(columns=['Type'])
y = data['Type']

print("Размер датасета:", X.shape)
print("Количество классов:", y.nunique())


# Кодирование целевых классов и стандартизация признаков

from sklearn.preprocessing import LabelEncoder
```

```
encoder = LabelEncoder()
y = encoder.fit_transform(y)

scaler = StandardScaler()
X = scaler.fit_transform(X)

print("Данные подготовлены.")
print("Количество признаков:", X.shape[1])
print("Количество классов:", len(np.unique(y)))


# Разделение данных на обучающую и тестовую выборки

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Обучающая выборка:", X_train.shape)
print("Тестовая выборка:", X_test.shape)


# Преобразование обучающих и тестовых данных в тензоры PyTorch

X_train = torch.tensor(X_train, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)

y_train = torch.tensor(y_train, dtype=torch.long)
y_test = torch.tensor(y_test, dtype=torch.long)

print("Тензоры PyTorch созданы.")
```

Результат:

```
Размер датасета: (214, 9)
Количество классов: 6

Данные подготовлены.
Количество признаков: 9
Количество классов: 6

Обучающая выборка: (171, 9)
Тестовая выборка: (43, 9)

Тензоры PyTorch созданы.
```

2. **Определение архитектуры нейронной сети**

- создайте класс, наследуемый от `torch.nn.Module`;
- в методе `__init__` определите все слои, которые будете использовать (например, `nn.Linear`, `nn.ReLU`, `nn.Dropout`);

• в методе `forward` опишите последовательность применения слоев к входным данным.

Код:

```
# Определение архитектуры многослойного перцептрона (MLP)

class MLP(nn.Module):
    def __init__(self):
        super(MLP, self).__init__()

        self.layers = nn.Sequential(
            nn.Linear(9, 10),
            nn.ReLU(),
            nn.Linear(10, 10),
            nn.ReLU(),
            nn.Linear(10, 6)
        )

    def forward(self, x):
        return self.layers(x)
```

3. **Инициализация модели, функции потерь и оптимизатора**

• создайте экземпляр вашей модели: `model = MLP();`

• определите функцию потерь. Для бинарной классификации используйте `nn.BCEWithLogitsLoss`, для многоклассовой – `nn.CrossEntropyLoss`;

• определите оптимизатор: `optimizer = torch.optim.Adam(model.parameters(), lr=0.001).`

Код:

```
# Инициализация модели, функции потерь и оптимизатора

model = MLP()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

4. **Написание цикла обучения (Training Loop)**

• запустите цикл на определенное количество эпох;

• внутри цикла:

1. переведите модель в режим обучения: `model.train();`
2. сделайте предсказание (forward pass): `y_pred = model(X_train);`
3. рассчитайте потери (loss): `loss = criterion(y_pred, y_train);`
4. обнулите градиенты: `optimizer.zero_grad();`

5. выполните обратное распространение ошибки: `loss.backward()`;
6. сделайте шаг оптимизации: `optimizer.step()`.

Код:

```
# Обучение нейронной сети с двумя скрытыми слоями

epochs = 100

for epoch in range(epochs):
    model.train()

    y_pred = model(X_train)
    loss = criterion(y_pred, y_train)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

print("Обучение завершено. Итоговая ошибка:", round(loss.item(),
4))
```

Результат:

```
Обучение завершено. Итоговая ошибка: 1.4681
```

5. **Оценка модели (Evaluation)**

- переведите модель в режим оценки: `model.eval()`;
- используйте `with torch.no_grad():`, чтобы отключить расчет градиентов;
- сделайте предсказания на тестовых данных;
- преобразуйте выходные данные (логиты) в предсказанные классы (например, с помощью `torch.argmax` или проверки порога `> 0`);
- рассчитайте метрики (`accuracy`, `f1-score` и т.д.), используя `sklearn.metrics`.

Код:

```
# Оценка модели на тестовой выборке

model.eval()

with torch.no_grad():
    y_pred = model(X_test)
    y_pred_classes = torch.argmax(y_pred, dim=1)

y_true = y_test.numpy()
y_pred_classes = y_pred_classes.numpy()
```

```
print("Accuracy:", round(accuracy_score(y_true, y_pred_classes),
4))
print("Precision:", round(precision_score(y_true,
y_pred_classes, average='weighted', zero_division=0), 4))
print("Recall:", round(recall_score(y_true, y_pred_classes,
average='weighted', zero_division=0), 4))
print("F1-score:", round(f1_score(y_true, y_pred_classes,
average='weighted', zero_division=0), 4))
```

Результат:

```
Accuracy: 0.3953
Precision: 0.2309
Recall: 0.3953
F1-score: 0.2685
```

Вариант 9 Определение вида стекла

- Glass Identification
- **Задача:** классифицировать тип стекла (многоклассовая задача).
- **Архитектура:**
  - входной слой;
  - два скрытых слоя по 10 нейронов в каждом (ReLU);
  - выходной слой с необходимым количеством нейронов (Softmax).
- **Эксперимент:** попробуйте обучить модель с одним скрытым слоем на 20 нейронов. Какая архитектура оказалась лучше?

Код:

```
# Определение экспериментальной нейронной сети с одним скрытым
слоем

class MLP_Experiment(nn.Module):
    def __init__(self):
        super(MLP_Experiment, self).__init__()

        self.layers = nn.Sequential(
            nn.Linear(9, 20),
            nn.ReLU(),
            nn.Linear(20, 6)
        )

    def forward(self, x):
        return self.layers(x)

# Инициализация и обучение экспериментальной модели

model_exp = MLP_Experiment()
criterion_exp = nn.CrossEntropyLoss()
```

```
optimizer_exp = optim.Adam(model_exp.parameters(), lr=0.001)

epochs = 100

for epoch in range(epochs):
    model_exp.train()

    y_pred = model_exp(X_train)
    loss = criterion_exp(y_pred, y_train)

    optimizer_exp.zero_grad()
    loss.backward()
    optimizer_exp.step()

print("Обучение экспериментальной модели завершено.")

# Оценка экспериментальной модели на тестовой выборке

model_exp.eval()

with torch.no_grad():
    y_pred_exp = model_exp(X_test)
    y_pred_exp_classes = torch.argmax(y_pred_exp, dim=1)

y_pred_exp_classes = y_pred_exp_classes.numpy()

print("Accuracy:", round(accuracy_score(y_true,
y_pred_exp_classes), 4))
print("Precision:", round(precision_score(y_true,
y_pred_exp_classes, average='weighted', zero_division=0), 4))
print("Recall:", round(recall_score(y_true, y_pred_exp_classes,
average='weighted', zero_division=0), 4))
print("F1-score:", round(f1_score(y_true, y_pred_exp_classes,
average='weighted', zero_division=0), 4))
```

Результат:

```
Обучение экспериментальной модели завершено.

Accuracy: 0.6047
Precision: 0.6165
Recall: 0.6047
F1-score: 0.5439
```

Лучше показала себя модель с одним скрытым слоем из 20 нейронов. У неё все показатели качества оказались выше, чем у первой модели с двумя скрытыми слоями по 10 нейронов. Поэтому можно сделать вывод, что в данном эксперименте вторая архитектура справилась с классификацией типов стекла лучше.

Вывод: построила, обучила и оценила многослойный перцептрон (MLP) для решения задачи классификации.